

JHQ-GPU 基线 JHQ_V14_STREAMING_ADD 对标 PVLDB 19(7):1530-1543 2026-08-31

把 JHQ-GPU 补成一篇能投的论文

CPU 版 JHQ 已经发在 PVLDB。GPU 版目前只有一条 recall-QPS 曲线。这份清单把「消融 + 系统指标 + 算法对比」拆成 10 组可执行实验，并指出两个必须先修的数据自洽性问题。

00 结论

现状：GPU 侧全部证据 = 6 个数据集 × 8 个 nprobe 点的 (recall, qps, build_time)，加上 VERSIONS.md 里 Vogue 单数据集、单 nprobe 的版本演进表。没有任何 GPU baseline、没有任何消融、没有任何系统指标。

教授给的三条（消融 / 系统参数 / 与其他算法比较）方向完全正确，但漏了两条更致命的：

你现在的「对比」全是 **CPU baseline**。results/cpu_*.csv 里只有 FAISS-IVFPQ / JQ+IVF / JHQ+IVF，全是 CPU。一篇 GPU 论文不跟 FAISS-GPU、cuVS IVF-PQ、CAGRA 比，第一轮审稿必挂。

现有数字自己跟自己打架。见下一节的两个 blocker。在修好之前，所有后续实验都是在沙子上盖楼。

工作量估计：基建 3-4 人日，P0 实验约 12-16 人日（含跑机时间），P1 约 8 人日。按每天能

跑进的基表，1/2 图可以列「消融实验方案」。

此/两/时/中/关，4-0/同/可以/到「仅/何/关键/元/田」。

已有证据	GPU BASELINE	消融实验	系统指标
~15%	0	0	0
相对 CPU 论文的实验体量	个已跑通	组 (CPU 论文有 4 组)	项 (PCIe/显存/占用率全空)

01 先修：两个数据自洽性问题

这两条不是「实验不够」，是「现有实验可能是错的」。写进论文会被 reviewer 一眼看出。

B1 · v14 在 Vogue 上比 v12 慢 2.4-3×，但 recall 逐位相同

jhq_v14_streaming_add 是 v12_transposed 的 fork，只改了 add()（流式编码以避免 17.8M×1024 撑爆显存），search 路径应当完全一致。arXiv 上确实一致（差 <2%），Vogue 上却差 3 倍：

数据集	NPROBE	RECALL	V12 QPS	V14 QPS	比值
arXiv-768	1	0.9431	323,385	328,641	1.02×
arXiv-768	8	0.9997	109,113	110,157	1.01×
Vogue-768	1	0.8862	49,123	20,690	0.42×
Vogue-768	4	0.9907	54,644	16,107	0.29×

recall 完全一致说明算法路径没变，所以这是环境或参数差异：不同机器 不同 batch size

GPU 未锁频、或那次跑的时候卡上有别的进程。文件名 `jhq_v14_vogue_test.csv` 的 `_test` 后缀也暗示这是一次随手测试。**动作：**用同一台机、锁频、同参数重跑 v12 与 v14 的 Vogue，确认两条曲线重合；不重合就要在代码里找真回归。

B2 · 规模与耗时不单调

搜索：Vogue (933K, d=768) 在 nprobe=1 只有 49K QPS, arXiv (2.25M, 同 d) 却有 323K QPS。规模大 2.4×, 反而快 6.6×。 `nlist` 差异 (1024 vs 4096) 只能解释 ~1.65×, 剩下 4× 无法解释。

建索引：BGE-M3 (10.1M) 用 52.1 s, Stella-TREC24 (17.8M, 同 d=1024) 只用 48.5 s。规模大 1.76×, 时间反而更短。

最可能的原因是这些 run 来自不同时间、不同参数 (M、batch_size、add_batch)、甚至不同机器，而 CSV 里只记录了 **nprobe**，没记录其余任何参数。**动作：**先做 E0.2 (结构化 run 记录)，然后所有论文数字统一重跑一遍。

02 缺口对照表

左边是 CPU 论文实际做了什么，右边是 GPU 侧现在有什么。一篇 GPU 论文不能比 CPU 论文的实验更薄。

实验维度	CPU 论文 (P1530-HAN)	JHQ-GPU 现状	状态	本清单编号
speed-accuracy 帕累托	Fig 4a: 6 数据集 × 7 baseline + brute-force	6 数据集 × 0 baseline	缺	E1
索引构建时间	Fig 4b: 6 数据集, 逐 baseline 对比	只有一个标量, 无分解、无对比	半	E1 · E8

索引 / 编码存储 (CR、ECL)	Table 3	无	缺	E8
距离误差界验证	Table 4	无 (可直接引用 CPU 论文结论)	可复用	—
消融: JL 变换	Fig 6(a,b)	无	缺	E2c
消融: 层次结构 (JQ vs JHQ)	Fig 6(c,d)	无	缺	E2c
消融: 标量 vs 向量量化	§5.3 case study	无	缺	E2c
敏感性: α	Fig 7	无	缺	E4
可扩展性 1M→17.8M	Fig 5	无 (只有 6 个独立数据集)	缺	E5
GPU 内核级消融	不适用	只有 Vogue 单点的 <code>VERSIONS.md</code> 表	半	E2a · E2b
PCIe / 显存 / 占用率	不适用	无	缺	E3 · E7
延迟分布 p50/p95/p99	不适用	无	缺	E6
能耗 / QPS-per-Watt	不适用	无	缺	E9
重复次数 / 方差 / 锁频	不适用	无 (单次 run)	缺	E0.3

03 基建：先写三个东西

不先做这一步，后面每组实验都要为了补一个指标而全部重跑。这是整份清单里投入产出比最高的 3 人日。

E0.1 GPU 遥测：NVML 采样线程 + 显存高水位

产出 `common/gpu_telemetry.cuh`：一个 RAII scope，进入时起后台线程按 10 ms 采样，退出时给出均值 / 峰值。

- 采什么
- `nvmlDeviceGetPcieThroughput` → PCIe RX / TX (KB/s)，配合 `nvmlDeviceGetCurrPcieLinkWidth/Generation` 算出链路利用率百分比 —— 这就是教授要的「PCI 使用率」
 - `nvmlDeviceGetUtilizationRates` → SM 利用率、显存控制器利用率
 - `nvmlDeviceGetPowerUsage` / `nvmlDeviceGetTemperature` / `nvmlDeviceGetClockInfo`（用来证明没降频）
 - `cudaMemGetInfo` 在 train / add / search 前后各取一次 → 显存高水位；host 侧读 `/proc/self/status` 的 `VmHWM` → 主机内存峰值

可抄 `/Users/apple/github/2020_song_ex/telemetry.h` 已经有类似结构，直接改。

E0.2 结构化 run 记录：一行 JSON = 一次实验的全部输入与输出

现在的 CSV 只有 `method,nprobe,recall,qps,build_time` —— 这正是 B2 无法归因的直接原因。改成每次 run 追加一行 JSON：

```
{
  "ts": "2026-08-31T10:22:03Z",
  "host": "gpu-01",
  "gpu": "RTX 5090",
  "driver": "...",
  "cuda": "12.6",
  "commit": "84266e5",
  "version": "jhq_v14_streaming_add",
  "dataset": "vogue-768",
  "n": 932328,
  "d": 768,
  "nq": 1000,
  "k": 10,
  "params": {
    "M": 96,
    "B": 8,
    "Br": 4,
    "alpha": 4.0,
    "nlist": 1024,
    "nprobe": 8,
    "batch_size": 256,
    "add_batch": 65536,
    "seed": 42
  },
  "clocks_locked": true,
  "reps": 5,
  "recall@10": 0.9982,
  "qps_median": 44955,
  "qps_iqr": [44120, 45330],
  "lat_ms": {
    "p50": 5.6,
    "p95": 6.1,
    "p99": 6.9
  },
  "build": 5,
  "train_ms": 812,
  "rotato_ms": ...,
  "kmeans_ms": ...,
  "encode_ms": ...,
  "total_ms": 288872
}
```

```
build_v2_train_mms .012, rotate_mms ...., kneads_mms ...., encode_mms ...., total_mms .200075,
"mem": {"vram_peak_mb": 11240, "index_mb": 9860, "host_rss_peak_mb": 3120},
"pcie": {"h2d_mb": 2.9, "d2h_mb": 0.08, "rx_util_pct": 1.4, "tx_util_pct": 0.2},
"power": {"avg_w": 218, "energy_per_query_mj": 4.85}}
```

然后所有画图脚本改成读这个 JSONL。已有的 `scripts/sweep_gpu.py` / `plot_v12_vs_cpu_all.py` 保留兼容层即可。

E0.3 测量纪律：锁频 + 重复 + 中位数

P0

`nvidia-smi -pm 1`、`nvidia-smi -lgc <固定SM频率>`、`-lmc` 锁显存频率。不锁频的 GPU 论文数字，reviewer 有权怀疑。

每个配置跑 **5 次**，报中位数 + IQR（现在是单次）。warm-up 1 次不计入（CUDA Graph 第一次调用要 capture）。

跑之前 `nvidia-smi --query-compute-apps` 确认独占卡。

固定 `seed=42`，并且把 groundtruth 生成方式写清楚（现在的 recall 计算在 `demo_*.cu` 里是 $O(nq \cdot k \cdot gt_k)$ 的朴素匹配，没问题，但要在论文里说明 $k=10$ 、 $gt_k=?$ ）。

04 E1 · 算法对比（最大缺口）

这是教授说的「与其他算法的比较」。核心原则：**GPU 方法必须跟 GPU 方法比**；跟 CPU 比只能作为附加的「加速比」论述，且 CPU 侧必须用多线程。

E1a GPU baseline 套件

P0

BASELINE	为什么必须有	对应 CPU 论文	你手上的现成资源
FAISS-GPU <code>GpuIndexIVFPQ</code>	最直接的同类对位：同 IVF、同 M、同 B=8。没有它就没有「我们比 PQ 快多少」这句话	PQ	<code>2021_faiss_gpu_ex/</code> 、 <code>faiss/</code>
FAISS-GPU <code>GpuIndexIVFFlat</code>	不压缩的上界参考，界定量化损失	—	同上
FAISS-GPU <code>GpuIndexFlatL2</code>	GPU 暴力搜索 = 精度上界 + 速度地板，CPU 论文里的 brute-force 对位	Brute-force	同上
cuVS / RAFT <code>IVF-PQ</code>	当前 GPU 量化索引的工业 SOTA，比 FAISS-GPU 强，不能只打 FAISS	—	<code>cuvs/</code>
CAGRA	最关键的一个。 GPU 图索引 SOTA。审稿人第一条意见一定是「为什么不跟 CAGRA 比」	—	<code>2024_cagra_repro/</code> (已有 <code>cagra_sweep.csv</code> 、 <code>cuvs_sweep.csv</code>)
GGNN 或 SONG	可选，补齐图方法谱系	—	<code>song/</code> 、 <code>2020_song_ex/</code>
BANG	只在做 out-of-core / 十亿规模那一节才需要	—	<code>2025_bang_repro/</code>
Extended-RaBitQ (GPU)	CPU 论文比了，GPU 版若有实现就补；没有就在 related work 里说明	RaBitQ-1/8bit	需自行找

公平性硬要求：所有 baseline 用同一台机、同一批 query、同一 groundtruth、同一 k=10，且都不做 rerank（跟 CPU 论文 §5 的设定保持一致，并在论文里明写）。各自扫自己的参数网格取帕累托前沿，不要只报一个点。

CPU 论文用的是**单线程搜索** (§5 实验平台明写)。如果 GPU 论文拿单线程 CPU 当分母去算加速比，reviewer 会直接判为不公平比较。

把 `scripts/bench_cpu_baselines.py` 扩成同时报 **1 线程 / 32 线程 / 全核 (96 核 192 线程)** 三档。

论文里两个数都给：「vs 单线程 JHQ-CPU 的 $N\times$ 」和「vs 96 核 JHQ-CPU 的 $M\times$ 」。后者才是有说服力的那个。

顺带给出 **吞吐/成本或吞吐/瓦**的归一化对比（见 E9），这是回应「GPU 当然更快」这类质疑的标准做法。

E1c 索引构建时间对比 —— 你最锋利的角度

P0

你现在的建索引时间是 **Vogue 933K / 26.7 s, arXiv 2.25M / 28.9 s, BGE-M3 10.1M / 52.1 s, Stella 17.8M / 48.5 s**（B2 修完后重测）。CAGRA 在千万级建图通常是分钟到十分钟量级，OPQ/LSQ++ 在 CPU 上更是小时量级。

JL 变换让码本训练几乎零成本，这条在 GPU 上比在 CPU 上更突出，因为 GPU 上其他方法的建索引开销无法被并行完全掩盖。建议把「构建时间 × 查询吞吐」的二维图做成论文的招牌图。

构建时间还要**分解**：JL 旋转 / Lloyd-Max 码本 / IVF k-means / 主码编码 / 残差码本 / 残差编码 / H2D 传输。这个分解同时喂给 E3 的 PCIe 分析。

05 E2 · 消融实验

教授说的「消融」在 GPU 论文里其实是三层，缺任何一层都会被追问。

E2a 累积瀑布：v1 → v14，在等召回下

P0

VERSIONS.md 已有一张表 (v1 1,070 → v12 44,890 QPS, 42×)，但它只在 Vogue 单数据集、单 nprobe=8，而且是 fixed-nprobe 而非 fixed-recall。论文里必须改成：

横轴：v1_plain → v3_ivf → v4_batched → v5_cuda_graph → v10_bytelut → v12_transposed → v14_streaming

纵轴：iso-recall QPS（分别在 recall@10 = 90% / 95% / 99% 三档插值）

至少 3 个数据集（小 / 中 / 大：Vogue、arXiv、Stella），证明结论不是单数据集偶然

v11_outerlut 作为「失败的尝试」保留一行，负结果反而增加可信度

E2b 留一消融：从 v14 逐个关掉优化

P0

累积瀑布有个已知弱点：顺序会影响归因。审稿人喜欢看 leave-one-out。建议不要再复制目录，改成在 jhq_v14_streaming_add 里加编译期开关：

```
-DJHQ_ABLATE_TRANSPOSE=0      # list_primary 退回 [N,M] 布局
-DJHQ_ABLATE_BYTELUT=0        # 退回 float LUT (重现 16-way bank conflict)
-DJHQ_ABLATE_CUDAGRAPH=0      # 逐 kernel launch
-DJHQ_ABLATE_BATCH=1          # batch_size=1
-DJHQ_ABLATE_ASYNC=0          # 可分页内存 + 同步 memcpy
-DJHQ_ABLATE_RESIDUAL=0        # 只用主码 = GPU 版 JQ
```

每关一个，报 iso-recall QPS 下降百分比，并配上 E7 的 profiler 证据（例如关掉 transpose 后

`sectors/request` 从 4 涨到 32, 大掉 byteLUT 后 shared BANK conflict 订致暴涨)。'性能数字 + 硬件计数器解释」的组合是这类论文最值钱的部分。

E2c 算法消融：对位 CPU 论文 Fig 6

P0

- JL 变换

关掉旋转矩阵 Π (`d_Pi_` 置为单位阵) $\rightarrow$ recall-QPS 曲线。CPU 论文 Fig 6(a,b) 的 GPU 复现。**额外的 GPU 论点**：不做 JL 时各子空间分布不均, byte LUT 的量化区间利用率下降, 可以用 profiler 显示 LUT 命中分布变差。
- 层次结构

JQ (只有主码) vs JHQ (主 + 残差精化)。CPU 论文的结论是「百万级 JQ 更快、千万级 JHQ 更好」; **GPU 上这个交叉点会移动** (残差精化是低并行度的 gather, GPU 上相对更贵), 这个「交叉点在 GPU 上前移/后移」本身就是一个可发表的观察。
- 标量 VS 向量量化

CPU 论文测出 285 $\times$ 差距。GPU 上重测, 预期差距更大或更小都值得报。
- IVF 有无

v1_plain (全库扫) vs v3_ivf。GPU 全库扫的相对代价比 CPU 低得多, 这是 GPU 特有的结论。
- CODEBOOK 训练量

训练样本 100K (当前 `n_train=min(nb,100000)`) vs 10K / 1M。证明 JL 让码本对训练样本量不敏感 —— 这是 JL 卖点的直接证据。

E2d v14 专属： `add_batch` 的构建时间 $\times$ 显存峰值帕累托

P0

`add_batch` (默认 65536) 是 v14 存在的**唯一理由**：v12 的 `add()` 会一次性物化两个 `n $\times$ d` float 缓冲 (按代码注释, Stella-TREC24 的 17.8M 行需要约 145 GB)。这条必须有实验支撑, 否则 v14 在论文里就是一个没有证据的工程改动。

扫 `add_batch  $\in$  {8K, 16K, 32K, 64K, 128K, 256K,  $\infty$ (=v12 行为)}`

据三条曲线：构建时间 显存高水位 H2D PCIe 累计流量

取一小段线：构建时间、显存占用率、H2D / D2H 字节数

标出 v12 在 BGE-M3 / Stella 上 OOM 的那个点 —— 这就是「v14 让单卡跑通 17.8M×1024」的证据

顺带回答：流式化是否付出了构建时间代价？（预期几乎没有，因为瓶颈在编码 kernel 不在传输）

06 E3 · 系统级参数（教授点名的部分）

「PCI 使用率、内存使用率」这类指标要落成一张每方法 × 每数据集一行的大表。下面是列的定义。

指标	怎么测	为什么审稿人在意
显存峰值（索引 / workspace 分开）	<code>cudaMemGetInfo</code> 在各阶段前后 + <code>nvidia-smi</code> 交叉验证	决定「这方法能在什么卡上跑多大数据」，是 GPU ANN 论文的核心约束
显存占用率（峰值 / 卡容量）	同上 ÷ 32 GB（RTX 5090）	直接回答「还能再大多少倍」
主机内存峰值	<code>/proc/self/status</code> 的 <code>VmHWM</code>	你用了 mmap 读 base（ <code>fvecs_mmap_io.cuh</code> ），这点要量化成卖点
PCIe H2D / D2H 字节数	代码内累计（query 批 = <code>nq×d×4</code> ，结果 = <code>nq×k×8</code> ）	解析值 + 实测值对上，说明没有隐藏传输
PCIe 链路利用率 %	NVML <code>PcieThroughput</code> ÷ 理论带宽（Gen×width）	教授点名要的指标，见下方说明
SM 利用率 / 显存控制器利用率	NVML <code>UtilizationRates</code> ，10 ms 采样	粗粒度，用来说明 GPU 没闲着

kernel 时间分解	CUDA event (jhq_v13_step_timing 已有框架) 或 nsys	指出瓶颈 kernel, 为 E2b 的每一项优化定位
DRAM 带宽达成率	ncu, 见 E7	证明 ADC scan 是访存受限、以及 transposed 布局把它推到多少

关于「PCI 使用率」的一个重要提醒

当前 JHQ-GPU v14 是全常驻设计: add() 之后码本、IVF、主码、残差码全在显存里, 查询期只有 nq×d×4 字节进、nq×k×8 字节出。以 Vogue 为例, 1000 条 query × 768 维 × 4 B ≈ 2.9 MB, 摊到毫秒级的查询上, PCIe Gen5 x16 的利用率会是百分之一二。

这不是实验失败, 这是一个结果 —— 而且是支持你设计的结果, 要主动写出来:「量化把 payload 压进显存, 使查询期 PCIe 完全不是瓶颈, 这正是我们相对 out-of-core 图方法的结构性优势。」

PCIe 真正有信息量的地方在另外两处, 这两处才应该占论文篇幅: (1) 构建期 —— E2d 的 add_batch 扫描, 流式编码把 H2D 从「一次性 145 GB 峰值」摊成稳定流; (2) 超显存场景 —— 如果要谈 100M/1B, 就要引入 hblock_v41 那条 wave-region streaming 路线, 那时 PCIe 利用率会成为真正的一等指标。要不要把这条放进本篇, 取决于论文范围 (见 §10)。

07 E4–E6 · 敏感性、可扩展性、延迟

E4 参数敏感性

P0

参数	扫描范围	对位
alpha	2.0 / 4.0 / 8.0 / $\alpha_{\max}=N/k$	CPU 论文 Fig 7, 必须复现
M	768d: 48/96/192/384; 1024d: 64/128/256; 1536d: 32/64/128/192/256; 3072d: 32...384	CPU 论文 §5 参数设定, 保持一致
[B, Br]	[8,4] / [4,8] / [8,8]	CPU 论文同款
nlist	1024 / 4096 / 16384 (B2 的根因之一, 必须统一)	CPU 论文固定 4096
nprobe	1...128 (已有)	已有
batch_size	1 / 8 / 32 / 128 / 256 / 1024 (GPU 特有)	见 E6
k	1 / 10 / 100	reviewer 常问 k=100 会不会崩

E5 可扩展性曲线

P1

CPU 论文 Fig 5 用 Stella-TREC24 的 1M / 5M / 10M 子集。GPU 版直接对位并加一档：**1M / 5M / 10M / 17.8M**，同一数据集、同一分布，报 QPS@90%recall、构建时间、显存峰值三条曲线。

顺带回答一个 GPU 特有的问题：**显存墙在哪里？**按当前编码率外推「单张 32 GB 卡最多能装多少向量」，并用 E2d 的 `add_batch` 数据说明构建端不会先撞墙。这句话是审稿人判断工作分量的关键。

E6 延迟分布与批大小

P0

这是 GPU ANN 论文最常被攻击的软肋。现在 `demo_jhq_v14_streaming_add.cu` 只测「1000 条 query 跑 5 遍取平均」，得出的是纯吞吐，没有延迟分布。

报 `batch_size ∈ {1, 8, 32, 128, 256, 1024}` 下的 QPS 与 p50 / p95 / p99 延迟，画成吞吐-延迟曲线

`batch_size=1` 的单查询延迟必须给 —— 在线服务场景就是这个数，很多 GPU 方法在这里输给 CPU

报 CUDA Graph 的首次 capture 开销（冷启动延迟）

把 `v7_spin_sync` 的作用在这里量化：自旋同步应该主要改善 p99，正好是它该出现的位置

08 E7-E9 · Profiler、存储、能耗

E7 Nsight Compute 硬件计数器

P1

对 5 个 kernel (`select_probes` / `build_byte_lut` / `scan_ivf_coalesced` / `residual_refine` / `batched_topk_final`) 各采一组：

```
ncu --kernel-name-base function --launch-count 3 \
  --metrics \
  sm__throughput.avg.pct_of_peak_sustained_elapsed,\
  gpu__dram_throughput.avg.pct_of_peak_sustained_elapsed,\
  sm__warps_active.avg.pct_of_peak_sustained_active,\
  lltex__t_sector_hit_rate.pct,\
  lts__t_sector_hit_rate.pct,\
  lltex__average_t_sectors_per_request_pipe_lsu_mem_global_op_ld.ratio,\
```

```
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum,\
smsp__sass_average_branch_targets_threads_uniform.pct \
./build/demo_jhq_v14_streaming_add ...
```

两个指标是 E2b 的直接证据，务必单独成表：

`average_t_sectors_per_request` —— **[M,N] 转置布局的量化证明**。`[N,M]` 布局下一个 warp 读 32 个向量的同一子空间会散落到 32 个 sector；转置后应降到 ~4。这就是 v12 那 1.7× 提升的物理解释。

`data_bank_conflicts_shared` —— **ByteLUT 的量化证明**。float LUT 的 16-way bank conflict 在这里应该是一个大数，byte LUT 后接近 0。这就是 v10 那 2× 提升的物理解释。

再补一张 **roofline**：证明 `scan_ivf` 是访存受限、且已经吃到 DRAM 峰值带宽的多大比例。这一张图能顶三段文字。

E8 存储与查询期访存量（对位 CPU 论文 Table 3）

P1

索引大小 / CR	按数据集报 MB 与相对原始 float32 的压缩比，和 FAISS-GPU IVFPQ、cuVS IVF-PQ、CAGRA（图 + 原始向量）并列。CAGRA 需要保留原始向量，这一栏你会赢得很难看，要放大。
ECL (EFFECTIVE CODE LENGTH)	CPU 论文定义 $ECL = L_p + (\alpha k/n) \cdot L_r$ 。GPU 上建议扩展成 "effective sectors per candidate"：因为 transposed 布局改变的不是读了多少 bit，而是这些 bit 分布在多少个 32 B sector 里。把 ECL 从「比特数」推广到「sector 数」是个小但真实的贡献，正好把 E7 的计数器和 CPU 论文的指标串起来。
显存驻留分解	码本 / IVF 质心 / 主码 / 残差码 / 校正项 / workspace 各占多少。让读者一眼看出「加大 n 时哪一项先爆」。

E9 能耗与成本归一化

P2

NVML 采功耗 → **QPS/W** 与 **每查询能耗 (mJ)**，和 96 核 EPYC 上的 JHQ-CPU 并列。这是回应「GPU 贵、比较不公平」的标准做法，做起来很便宜 (E0.1 已经在采功耗了)，性价比高。可选再加一条 **QPS/\$** (按云上单价)。

09 论文图表映射

把上面的实验直接排成论文的 §5，方便你判断哪些必须做完。

论文位置	内容	来自	优先级
Table 2	数据集统计 (N, D, LID, RC) + 硬件与软件版本	沿用 CPU 论文 + E0.2	P0
Fig 4	recall-QPS 帕累托, 3 主数据集 × 全部 GPU baseline	E1a	P0
Fig 5	构建时间 × 查询吞吐 二维图 (招牌图)	E1c	P0
Table 3	系统指标总表: 显存、主机内存、PCIe、SM 利用率、功耗	E3 + E9	P0
Fig 6	优化瀑布 (v1→v14, iso-recall, 3 数据集)	E2a	P0
Table 4	留一消融 + 对应硬件计数器	E2b + E7	P0
Fig 7	算法消融: JL / 层次 / SQ-vs-VQ	E2c	P0
Fig 8	<code>add_batch</code> : 构建时间 × 显存峰值帕累托 + v12 OOM 点	E2d	P0

Fig 9	吞吐-延迟曲线 (batch 1→1024, p50/p95/p99)	E6	P0
Fig 10	可扩展性 1M→17.8M (QPS / 构建时间 / 显存)	E5	P1
Fig 11	α 敏感性	E4	P0
Table 5	索引存储与 ECL / sectors-per-candidate	E8	P1
Fig 12	roofline + kernel 时间分解	E7	P1
Appendix	其余 3 个数据集的全部曲线、M / [B,Br] / nlist / k 敏感性	E4	P1

10 一个必须先想清楚的问题：论文的卖点是什么

教授列的三项是**实验完备性**的要求。但实验做满不等于能中 —— 还得回答「把 JHQ 搬上 GPU，新在哪」。这一节是我的判断，建议你拿去和老师对一次。

V1→V14 的改动	性质	能不能当贡献
JL 变换 → 训练无关码本 → 秒级建索引	算法 × 硬件 协同	强 对 CAGRA / OPQ / LSQ++ 的建索引成本差是数量级的
<code>list_primary</code> 转置成 <code>[M, N]</code>	访存布局	强 但必须用 E7 的 sectors/request 把它讲成「JL 的子空间独立性使这种布局成为可能」，而不是一句「我们转置了」
Byte LUT <code>[B,M,256]</code> 消除 bank conflict	访存布局	中 依赖「B=8 位量化让 LUT 能塞进 byte」这个 JL 特有前提，要这么讲
流式 <code>add()</code> (v14)	系统	中 有了 E2d 的证据就是「单卡跑通 17.8M×1024」的支撑
CUDA Graph / pinned asvnc / spin sync	标准工程	弱 放在瀑布图里作为工程基线即可，不要写成贡献

```
... / batching
```

建议的主线

「训练无关的量化让 GPU 索引构建成本趋近于零，同时其分布性质使得 ADC 扫描能做到完全合并访存」——一句话把 JL（算法）和转置布局 + byte LUT（硬件）绑成一个因果链，而不是并列的两个技巧。E1c（建索引对比）+ E7（sectors/request）是这条主线的两根支柱。

范围决策：要不要把 hblock 放进来

仓库里其实有两条独立的线：`jhq_v1...v14`（IVF + 量化，全常驻）和 `hblock_v1...v43`（块级图 + 区域流式，面向超显存）。建议这一篇只写 **jhq 线**，理由是：(1) 它和 CPU 论文一一对位，实验设计可以直接照搬；(2) hblock 的 SPACEV-100M 召回目前还只有 0.53@ef=256，离能投的水平有距离。hblock 留作第二篇（out-of-core GPU ANN），那篇里 PCIe 利用率才是主角。这个取舍需要老师拍板。

11 执行顺序

周	做什么	产出
W1	B1 / B2 排查复现；E0.1 遥测；E0.2 JSONL；E0.3 锁频与重复	可信的测量底座；6 数据集全部重跑一遍
W2	E1a FAISS-GPU + cuVS + CAGRA 跑通；E1b CPU 多线程	Fig 4 的全部曲线
W3	E2b 编译期消融开关；E2a 瀑布重跑；E2d <code>add_batch</code> 扫描	Fig 6 / Table 4 / Fig 8
W4	E2c 算法消融；E4 敏感性；E6 延迟分布	Fig 7 / Fig 9 / Fig 11

W5	E3 系统指标总表；E7 ncu + roofline；E1c 构建时间分解	Table 3 / Fig 5 / Fig 12
W6	E5 可扩展性；E8 存储；E9 能耗；补附录	Fig 10 / Table 5 / Appendix

如果时间只够一半：砍掉 E5、E8、E9 和 E7 的 roofline，但 **E0（基建）、E1a（GPU baseline，尤其 CAGRA）、E2b（留一消融）、E3（系统指标）、E6（延迟）** 这五项一项都不能砍——它们分别对应「可信」「公平」「归因」「教授要求」「常见攻击点」。

依据：`p1530-han.pdf`（PVLDB 19(7):1530–1543）§5 实验设计；仓库 `JHQ_GPU` 的 `VERSIONS.md`、`results/*.csv`、`jhq_v14_streaming_add/`、`scripts/`；CPU 侧 `JHQ_repro/`。所有性能数字均引自 `results/` 下已有 CSV，尚未重新验证——见 §01。